

SKILLCOMPOSER: Learning Reusable Skills for Natural-Language Robot Programming

John Woods¹ and Hasti Seifi¹

Abstract—Natural-language interfaces can lower the barrier to programming robots, but existing systems struggle when users request complex tasks. While large language models (LLMs) perform well with simple commands, they often struggle to generate code for multi-step tasks, decompose high-level instructions, or reuse prior solutions. We present SkillComposer, an interactive natural-language robot programming system for simulation environments that continually learns reusable program abstractions. SkillComposer uses a generate-test architecture in which an LLM iteratively generates and revises robot programs before execution. Successful programs are stored and processed by an online library-learning algorithm that compresses recurring function sequences into reusable macro skills for future tasks. We evaluate SkillComposer through ablation experiments and a user study with 12 participants to determine its effectiveness on manipulation and robot caregiving tasks. The results show that evaluator-guided generation and learned abstractions improve success rates and usability while reducing user effort in natural-language robot programming.

I. INTRODUCTION

Robotics simulators increase the safety and efficiency of designing, testing, and refining robot behaviors before deployment in the physical world [1], [2], [3]. However, simulators still require users to understand programming and robotics, creating a barrier for domain experts who know what behavior they want but lack the technical expertise to implement it. This challenge is especially visible in assistive robotics, where caregivers, clinicians, and researchers may want to specify tasks involving object manipulation, feeding, or other human-robot interaction (HRI) tasks [4]. RCareWorld [5], a Unity-based caregiving robot simulator with Python and Robot Operating System (ROS) interfaces, provides a useful testbed for these tasks, but specifying robot behavior remains a programming-heavy process.

Recent advances in Large Language Models (LLMs) have made natural-language robot programming increasingly practical, allowing users to describe tasks in ordinary language rather than writing low-level robot code [6], [7], [8]. Prior systems have shown that LLMs can select robot actions, generate executable programs, and help non-expert users specify robotic tasks [9], [10]. However, these systems typically rely on a fixed set of manually defined primitive skills, which can require users to decompose complex goals, repair failed generations, or adapt their language to the system’s available API. Most also treat each prompt as an independent interaction, rather than learning reusable skills from recurring program structure.

To address these limitations, we introduce SkillComposer, an interactive natural-language robot programming system that learns reusable robot skills from generated programs. Rather than treating each user prompt as an isolated generation task, the system generates, evaluates, and stores successful robot programs for future reuse. A generate-test loop uses a coder LLM to produce candidate programs and an evaluator LLM to provide feedback before a program is accepted. Accepted programs are added to an experience buffer and processed by the online library-learning algorithm Stitch [11], which identifies recurring code structures and compresses them into reusable macro skills. These macros are provided to the coder model for subsequent prompts, allowing the system to reuse learned skills instead of repeatedly generating the same low-level function sequences.

To assess SkillComposer, we ask two research questions: **RQ1:** Does combining generate-test with library learning improve performance across manipulation and interaction tasks? **RQ2:** How does SkillComposer affect usability and user experience compared to a baseline coding LLM? We answer RQ1 with ablation experiments across four system variants in generic manipulation and assisted feeding scenes, measuring task success, program length, runtime, and macro usage. To answer RQ2, we conduct a user study with 12 participants comparing a baseline coding LLM against SkillComposer. We measure task completion rate and time as participants complete object rearrangement, meal preparation, and care-recipient feeding tasks, followed by usability ratings and qualitative feedback.

Overall, this paper contributes: (i) SkillComposer, an interactive natural-language robot programming system that learns reusable robot skills from generated robot programs, and (ii) empirical results from ablation experiments and a user study demonstrating how generate-test and online skill learning affect task success, efficiency, skill reuse, and usability in generic manipulation and assisted feeding scenarios. Together, these contributions show how library-learning techniques from programming languages research can enable natural language robot programming systems to move beyond one-off code generation toward interfaces that retain, reuse, and refine task knowledge over repeated use.

II. RELATED WORK

A. Natural Language Robot Programming

LLMs have increasingly been explored as natural language interfaces for robot programming. SayCan selects robot actions from language instructions based on the robot’s physical capabilities [8], while ProgPrompt generates robot

¹School of Computing and Augmented Intelligence, Arizona State University, Tempe, AZ {jmwood23, hasti.seifi}@asu.edu

programs from available actions, objects, and example programs [7]. ChatGPT for Robotics demonstrates how general-purpose LLMs can generate robot code and task plans when provided with the robot API in the prompt [6], and GenSim uses LLMs to generate code for robotic tasks and saves high-quality generations to a library for future model fine-tuning and evaluation [12]. Recent systems have also focused on making robot programming more accessible for end users. Alchemist uses LLMs to enable novice and expert users to program robot pick-and-place tasks using natural language [9], but they report that LLM-generated code is often unreliable and that effective prompting remains challenging for novices. Cocobo pairs LLMs with an interactive diagram interface to enable modification and debugging of programs [10], although some users found the interface challenging to use without programming knowledge. Together, these systems demonstrate the potential of natural language as an interface for robot programming, but they generally rely upon a fixed set of manually provided skills [6], [7], [8]. In contrast, SkillComposer focuses on enabling the programming interface to improve over time by learning reusable macro skills from previously generated programs.

Several works have also investigated methods for improving the reliability of LLM-generated code. Code-generation benchmarks emphasize that generated programs should be evaluated for task correctness in addition to syntactic validity [13]. MultiTalk uses a generate-test loop in which a planning LLM iteratively refines generated robot programs based on feedback from an analyzer LLM and visual input [14]. SkillComposer builds on this line of work through a generate-test loop that combines LLM-based functional evaluation with deterministic syntax validation before execution. Unlike prior approaches [14], accepted programs are retained and used as a growing corpus for online macro learning, allowing the system to improve future generations rather than only the current one.

B. Learning Reusable Robot Skills and Program Abstractions

Many recent studies have explored how agents and robots can expand their capabilities through reusable tools or skills. In LLM agent settings, LATM [15] and CREATOR [16] use LLMs to generate new callable tools via prompting rather than extracting them from previously generated programs, while Voyager continually builds a library of executable skills through interaction with the Minecraft environment [17]. In robotics, LLRL uses LLMs and a wake-sleep paradigm to build a library of reusable manipulation skills with guidance from a human teacher [18], and HALP uses an LLM to identify when robots should learn new skills from human interaction [19]. LYRA extends LLM-based robot task generation to long-horizon settings, but relies on users to design a learning curriculum and guide skill acquisition [20]. While prior robotics systems demonstrate that reusable skills can support long-horizon task execution, skill acquisition often depends on explicit human teaching, curriculum design, or interaction with the environment [18], [19], [20]. In

contrast, SkillComposer learns reusable robot skills directly from accepted LLM-generated robot programs, allowing the callable skill set to grow from the user’s interaction history without requiring users to manually design new skills.

Outside of robotics, research in the programming languages community investigates how systems can learn reusable abstractions from existing programs. DreamCoder learns reusable library functions from a corpus of programs using a wake-sleep paradigm to improve future program synthesis [21]. Stitch later introduced an efficient library learning algorithm that discovers abstractions to minimize program corpus size [11]. Others have combined library learning with language models. LAPS builds upon DreamCoder and uses natural language to guide abstraction learning and program synthesis [22], while LILO combines Stitch with LLM-guided synthesis to create documentation of learned abstractions [23]. Yet, these approaches focus on functional program synthesis over fixed corpora, whereas SkillComposer adapts library learning to imperative robot programs generated during interactive natural-language programming.

III. SKILLCOMPOSER

SkillComposer is an interactive natural language robot programming system that combines two mechanisms: (i) a generate-test loop for producing executable robot programs from user prompts, and (ii) an online macro learning module that discovers reusable skills from previously accepted programs. The full architecture is shown in Figure 1. Users interact with the system through a locally hosted Gradio interface, shown in Figure 2, which supports natural language prompting and manual robot and camera control. The interface displays the RCareWorld [5] simulation and conversation history, which includes a summary of generated function calls and a natural-language explanation after each prompt, allowing users to inspect which skills the system uses.

A. Generate-Test Loop

The Generate-Test Loop includes a coder LLM to generate candidate robot programs and a two-stage evaluation pipeline for checking functional and structural validity of programs before execution.

a) Coder LLM: Given a user instruction, SkillComposer first sends the prompt to the coder LLM, which generates a candidate robot program from the available function schemas. The system starts out with five primitive functions: `get_info`, `move_to_object`, `move_to_position`, `grasp_object`, and `release_object`. Programs are represented as JSON function calls, where each call specifies a function name and a dictionary of argument values. This structured representation allows generated programs to be parsed, validated, and executed as robot actions. Using schema-constrained JSON distinguishes SkillComposer from approaches that ask LLMs to generate free-form code, since generated programs can be easily checked against the available robot API before execution.

SkillComposer

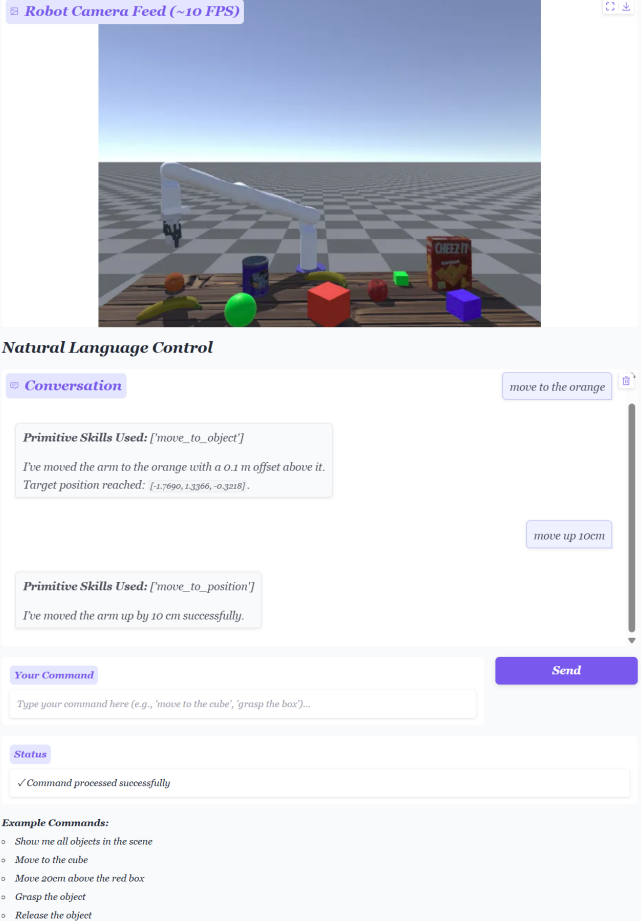


Fig. 2: The SkillComposer user interface, which contains a live camera feed from the simulation and a chat interface for interacting with the system.

amount of valid candidate abstractions before filtering. The expansion procedure also introduces fresh variables for any missing arguments, converting partial function applications learned by Stitch into valid robot programs.

The resulting abstractions are then checked for structural validity. A valid macro must contain at least two primitive function calls, use `then` only as a binary sequencing operator, use only known primitive functions, and place variables only in primitive argument positions. These constraints filter out abstractions that may be syntactically valid for Stitch but do not correspond to executable imperative robot programs. Finally, variables are renamed consistently so duplicate abstractions can be identified and removed.

Because many high-ranking Stitch abstractions are invalid under these constraints, SkillComposer reimplements the outer Stitch learning loop to select only valid abstractions at each iteration. After filtering, SkillComposer selects the highest-ranked valid candidate and rewrites the corpus using that abstraction. This repeats until either ten valid abstractions have been learned or Stitch produces no additional valid candidates. The macro set is relearned after each accepted

JSON Code

```
{
  "function": "grasp_object",
  "args": {
    "name": "Knife",
    "lift_height": 0.5
  }
},
{
  "function": "move_to_position",
  "args": {
    "x": -0.4,
    "y": 0.0,
    "z": -0.3,
    "relative": true
  }
},
{
  "function": "release_object",
  "args": {}
}
```

s-Expression

```
(then (grasp_object Knife 0.5) (then
  (move_to_position -0.4 0.0 -0.3 2.0 true) (release_object true 0.1))
)
```

Fig. 3: An equivalent portion of code in the JSON representation (top) and s-expression representation (bottom). Default arguments are added into the s-expression based on the defaults listed in the function schema.

prompt, so the available skills always reflect the user’s full interaction history.

c) *Generating a Function Schema*: Each valid abstraction is converted into a function schema for use with the coder and evaluator LLMs. The schema specifies the macro name, description, parameters, parameter types, and parameter descriptions. Parameter names and types are inferred by tracing abstraction variables to their corresponding primitive function arguments. The schema template is then sent to a documentation LLM, along with the macro body and contextual examples from user prompts and generated programs in which the abstraction appears. The documentation LLM generates a human-readable skill name, skill description, and parameter descriptions, which are added to the macro schema. This allows the schema to reflect both the learned program structure and the user’s natural-language prompts.

d) *Using Learned Macros*: The completed schema is added to the same function schema store as primitive functions, so future coder and evaluator prompts include both primitives and learned macros. When generating future programs, the coder LLM may call a learned macro if deemed relevant to the user’s prompt. Before execution, macro calls are expanded back into primitive robot calls, preserving simulator compatibility while allowing the LLM to generate programs at a higher level of abstraction. After execution, the conversation UI shows the skills used and a natural language summary of the output.

IV. EXPERIMENTS

We evaluate SkillComposer through two complementary studies. First, we conduct ablation experiments to isolate the effects of the generate-test loop and online macro learning on task performance (RQ1). Second, we conduct a within-subjects study to compare the user experience of SkillComposer against a baseline coding LLM in an interactive robot programming setting (RQ2).

A. Ablation Experiments

To evaluate the contribution of each component of SkillComposer (RQ1), we perform ablation experiments across four system variants: (1) *Coder LLM* where the LLM generates robot programs directly from user prompts, (2) *Coder LLM + Eval LLM* includes the generate-test loop, allowing candidate programs to be revised using evaluator feedback before execution, (3) *Coder LLM + Stitch* includes online macro learning, allowing the coder LLM to use previously

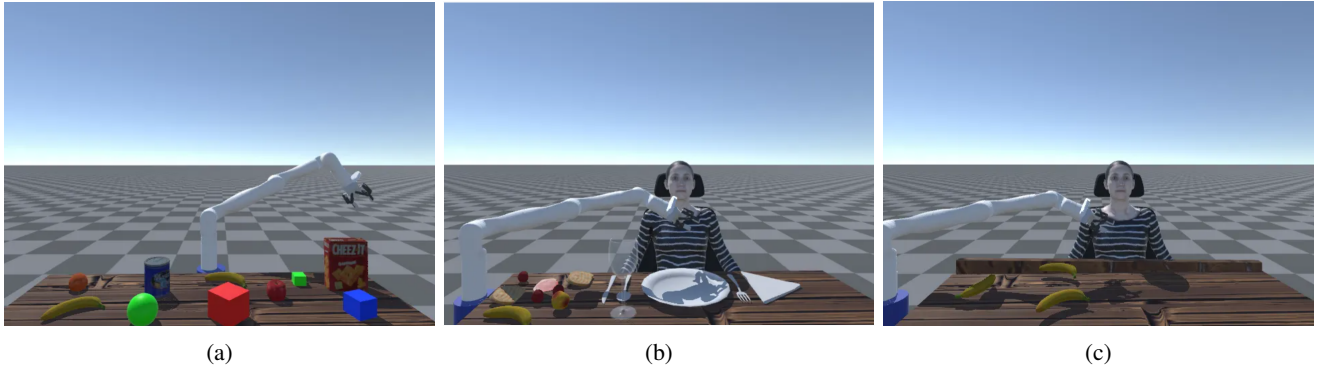


Fig. 4: Three simulation environments used in the experiments. (a) Objects: evaluates object manipulation, spatial reasoning, multi-step actions, and goal planning (prompts 1–20). (b) Feeding: evaluates simple and multi-step HRI tasks (prompts 21–40) and is used in the main user study. (c) Bananas: used for the user study practice task.

learned macro skills, but does not use evaluator-guided revision, and (4) *SkillComposer* combines the generate-test loop with online macro learning.

Each variant is evaluated in two RCareWorld environments. The Objects environment evaluates basic object manipulation, spatial reasoning, multi-step action planning, and goal planning (Figure 4a). The Feeding environment evaluates assistive caregiving tasks involving both simple and multi-step Human-Robot Interaction (Figure 4b). For each environment, we use a set of 20 natural-language prompts covering the aforementioned areas.

We measure four dependent variables: (1) *success rate* measures whether the generated program successfully completes the requested prompt in simulation, (2) *program length* counts the number of function calls in the final generated program as a measure of program complexity, (3) *macro usage* measures the percentage of program length that consists of macro calls, and (4) *runtime* measures the time required to produce an accepted program, including any generate-test iterations when applicable. Because LLM outputs are stochastic, each system-environment pair is run five times, and results are averaged across trials.

B. User Study

To evaluate the user experience of SkillComposer (RQ2), we ran a within-subjects user study comparing the full SkillComposer system with a baseline coding LLM, similar to the one used in prior work [24]. The baseline condition consisted of the coder LLM, while the SkillComposer condition included both the generate-test loop and online macro learning. The user-facing interface was identical across conditions, and backend order was counterbalanced across participants to reduce order effects. Participants received a \$15 Amazon gift card for their time. The study was approved by the institutional review board (IRB).

We recruited 12 participants (3 female, 9 male, 23–67 years old). Participants had upper-intermediate English proficiency, normal or corrected-to-normal vision, and none or beginner-level experience with robotics or programming. These crite-

ria targeted novice end users likely to benefit from natural-language robot programming.

Each study session lasted about 75 minutes. Participants first received an overview of the study, provided written consent, and completed a background questionnaire collecting demographic information and prior experience with robotics and programming. They then completed a short practice task (moving a banana 25 cm to the right) in the Bananas environment (Figure 4c) to become familiar with the interface.

Next, participants completed tasks using both systems. For each system, participants worked in the Feeding environment and completed three high-level robot programming tasks: rearranging objects on the table, preparing a meal, and feeding the care recipient. Participants wrote their own natural-language prompts and could send as many prompts as desired. We collected both objective and subjective measures. Objective measures included the number of prompts used, task success rate, and task completion time. These metrics assessed how well participants could complete the above high-level tasks using the system. Subjective measures included the System Usability Scale (SUS) [25] and 7-point Likert ratings of the system’s effectiveness in understanding commands, output predictability, alignment of robot skills with the task, and overall satisfaction.

At the end, participants engaged in a semi-structured interview. The interview asked participants to compare the two systems, describe which system they preferred, identify aspects that were helpful or confusing, and discuss whether either system seemed to adopt or reuse useful behaviors over time. Participants were also asked about the meaningfulness of learned skills, prompting strategies they used, types of tasks the systems handled well or poorly, and suggestions for improvement. The interview responses were audio-recorded and later transcribed for analysis.

V. RESULTS

A. Ablation Experiments

Table I summarizes the ablation results for each prompt category. The generate-test loop produced the largest im-

TABLE I: Summary of ablation experiments for each prompt category, averaged over all trials and environments. A representative prompt from each category is provided for reference.

Prompt Category	Example Prompt	System	Success Rate	Program Length	Macro Usage (%)	Runtime (s)
Object Manipulation	<i>Grab a yellow fruit</i>	Coder LLM	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	–	2.26 $\pm$ 0.80
		Coder LLM + Eval LLM	0.96 $\pm$ 0.09	1.32 $\pm$ 0.44	–	9.58 $\pm$ 9.86
		Coder LLM + Stitch	1.00 $\pm$ 0.00	1.08 $\pm$ 0.11	0.00 $\pm$ 0.00	2.24 $\pm$ 0.67
		SkillComposer	0.96 $\pm$ 0.09	1.32 $\pm$ 0.36	8.00 $\pm$ 17.89	13.97 $\pm$ 9.80
Spatial Reasoning	<i>Move rightmost banana to the left of the can</i>	Coder LLM	0.24 $\pm$ 0.43	0.96 $\pm$ 0.41	–	11.94 $\pm$ 4.05
		Coder LLM + Eval LLM	0.76 $\pm$ 0.43	2.82 $\pm$ 0.76	–	50.43 $\pm$ 17.02
		Coder LLM + Stitch	0.24 $\pm$ 0.43	1.00 $\pm$ 0.58	0.00 $\pm$ 0.00	14.51 $\pm$ 5.80
		SkillComposer	0.40 $\pm$ 0.42	2.06 $\pm$ 0.90	37.33 $\pm$ 43.87	30.47 $\pm$ 18.28
Multi-Step Actions	<i>Pick up the sphere, move it to the right 50cm, then release it</i>	Coder LLM	0.32 $\pm$ 0.27	1.68 $\pm$ 0.46	–	7.21 $\pm$ 2.55
		Coder LLM + Eval LLM	1.00 $\pm$ 0.00	3.60 $\pm$ 1.52	–	22.51 $\pm$ 11.81
		Coder LLM + Stitch	0.32 $\pm$ 0.23	1.80 $\pm$ 0.71	0.00 $\pm$ 0.00	9.38 $\pm$ 3.28
		SkillComposer	0.96 $\pm$ 0.09	2.32 $\pm$ 1.10	45.33 $\pm$ 51.08	20.59 $\pm$ 10.74
Goal Planning	<i>Stack all the cubes</i>	Coder LLM	0.08 $\pm$ 0.18	1.72 $\pm$ 2.06	–	11.67 $\pm$ 3.62
		Coder LLM + Eval LLM	0.68 $\pm$ 0.39	13.07 $\pm$ 11.21	–	50.33 $\pm$ 26.83
		Coder LLM + Stitch	0.00 $\pm$ 0.00	0.64 $\pm$ 0.22	0.00 $\pm$ 0.00	16.50 $\pm$ 5.07
		SkillComposer	0.76 $\pm$ 0.33	6.17 $\pm$ 4.48	86.00 $\pm$ 21.91	49.18 $\pm$ 9.25
Human-Robot Interaction	<i>Put the fork up to the user's mouth</i>	Coder LLM	0.14 $\pm$ 0.19	1.16 $\pm$ 0.32	–	15.96 $\pm$ 4.33
		Coder LLM + Eval LLM	0.94 $\pm$ 0.13	3.44 $\pm$ 1.05	–	47.49 $\pm$ 15.50
		Coder LLM + Stitch	0.16 $\pm$ 0.18	1.16 $\pm$ 0.23	4.00 $\pm$ 8.43	10.71 $\pm$ 2.98
		SkillComposer	0.86 $\pm$ 0.23	2.86 $\pm$ 1.04	22.00 $\pm$ 23.94	33.36 $\pm$ 12.14
Multi-Step Human-Robot Interaction	<i>Feed the peach first and then the banana to the user</i>	Coder LLM	0.02 $\pm$ 0.06	1.46 $\pm$ 1.07	–	16.06 $\pm$ 2.50
		Coder LLM + Eval LLM	0.98 $\pm$ 0.06	10.39 $\pm$ 6.05	–	53.95 $\pm$ 11.09
		Coder LLM + Stitch	0.04 $\pm$ 0.08	1.52 $\pm$ 0.90	5.00 $\pm$ 10.54	11.77 $\pm$ 3.60
		SkillComposer	1.00 $\pm$ 0.00	5.50 $\pm$ 1.75	55.82 $\pm$ 34.06	27.98 $\pm$ 11.37

Macro Usage by Prompt

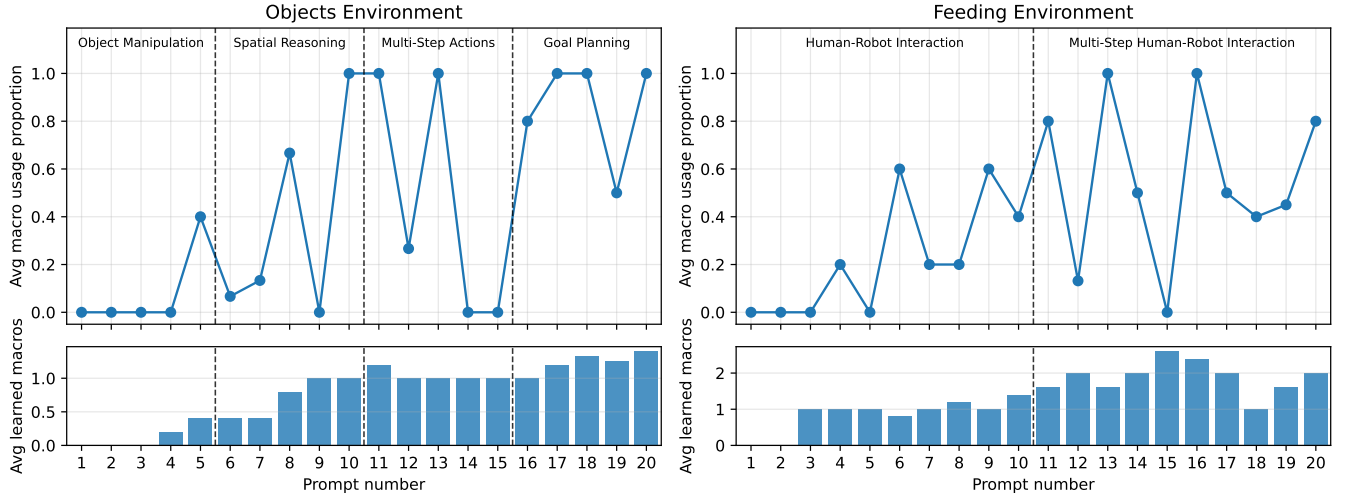


Fig. 5: Top: Average SkillComposer macro usage across five runs in the Objects (left) and Feeding (right) environments versus the number of input prompts. Bottom: Average number of learned macros across five runs versus input prompts.

provement in success rate, with evaluator-guided systems outperforming direct code generation across most categories. The exception was basic object manipulation, where all variants performed well, suggesting that direct LLM generation is often sufficient for simple commands. However, the generate-test loop also introduced the largest latency cost, increasing generation time by approximately four to five times due to additional evaluation and revision steps.

Macro learning primarily affected program complexity rather than success rate. Systems with macro learning generated shorter programs on average, indicating that learned macros successfully compressed repeated low-level action

sequences into higher-level function calls. Macro learning alone did not substantially improve success over the baseline though, suggesting that learned macros are most effective when paired with the generate-test loop. Overall, the generate-test system achieved the highest success rate, while the full SkillComposer system had comparable performance with more compact programs and visible macro reuse.

We also analyzed macro usage over time in SkillComposer. Figure 5 shows the average proportion of macros in generated programs across the 20 prompts in each environment, along with the average number of learned macros available at each point. In both environments, macro avail-

TABLE II: Summary of user study task metrics, averaged over all participants.

System	Task	Success Rate	Num. Prompts	Task Time (m:ss)
Baseline	Rearrange Objects	0.50 $\pm$ 0.52	5.8 $\pm$ 2.2	6:09 $\pm$ 2:57
	Meal Preparation	0.92 $\pm$ 0.29	8.2 $\pm$ 5.6	6:10 $\pm$ 3:19
	Feed Care Recipient	0.75 $\pm$ 0.45	6.6 $\pm$ 2.7	3:48 $\pm$ 1:43
SkillComposer	Rearrange Objects	0.92 $\pm$ 0.29	1.9 $\pm$ 1.2	3:20 $\pm$ 1:20
	Meal Preparation	0.92 $\pm$ 0.29	3.2 $\pm$ 2.6	3:59 $\pm$ 1:48
	Feed Care Recipient	0.67 $\pm$ 0.49	3.5 $\pm$ 1.6	4:16 $\pm$ 1:57

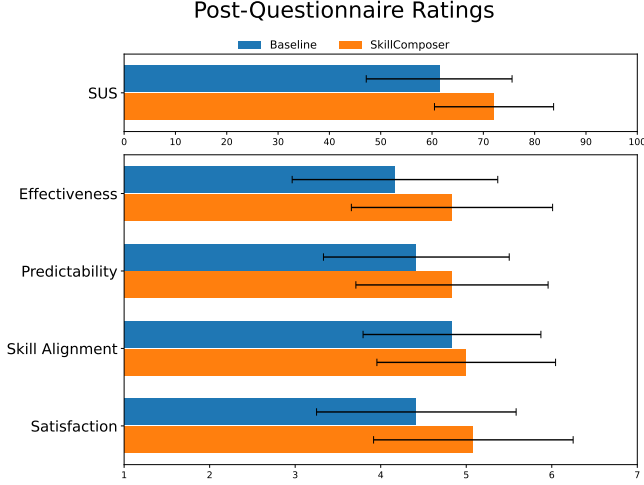


Fig. 6: Post-questionnaire results from the user study. SUS is measured on a 100-point scale [25], while the other four questions are measured on a 7-point Likert scale, with 7 being the highest. Error bars report $\mu \pm 2\sigma$.

ability generally increased as more accepted programs were added to the corpus, and macro usage tended to increase as more learned skills became available. The learned macros captured recurring action patterns in each environment. Examples include `move_and_place_object` in the Objects environment, and `place_object_on_plate` and `hand_object_to_user` in the Feeding environment.

B. User Study

Table II summarizes the task-level results from the user study. SkillComposer achieved equal or higher task completion rates than the baseline system on the Rearrange Objects and Meal Preparation tasks while requiring fewer prompts on average across all three tasks. SkillComposer also reduced average task completion time for the Rearrange Objects and Meal Preparation tasks. The exception was the Feed Care Recipient task, where the baseline achieved a slightly higher success rate and lower completion time. This task often required small corrective movements near the end of execution, making rapid interaction more beneficial than higher-level task planning.

The largest difference between the systems was observed in the Rearrange Objects task, where the baseline achieved substantially lower task success. Participants tended to give high-level instructions involving multiple objects (e.g. P10

prompt “*I want you to swap the positions of the knife and the spoon.*”), whereas the baseline often required users to decompose these requests into basic prompts focusing on one object or action at a time. As participants gained experience with the baseline system, they gradually adapted their prompting strategies to step-by-step commands, reducing this performance gap in the next two tasks.

Figure 6 summarizes participants’ subjective ratings. Overall, participants reported a more favorable perception of SkillComposer across all subjective measures. The largest effect was observed for SUS, where SkillComposer scored 10.7 points higher on average, indicating a moderate effect (Cohen’s $d = 0.46$). SkillComposer achieved a mean SUS score of 72.1 ($\sigma = 20.1$), categorized as good usability [26]. The baseline only achieved a mean SUS score of 61.4 ($\sigma = 24.6$), categorized as OK usability [26]. However, paired-samples t -tests found no statistically significant difference between systems in any subjective metrics (all $p > 0.05$).

The qualitative responses provide additional insight into these trends. 11 of the 12 participants preferred SkillComposer over the baseline, often noting its stronger support for high-level, multi-step instructions and lower effort. P7 commented that “*In [SkillComposer], I could use a single prompt. Whereas in [the baseline], I have to break down one task into six or seven prompts to get it to work.*” Similarly, P8 said that “*It feels like [SkillComposer] understands me. But the [baseline] feels more like a robot. I have to kind of program a little bit.*” Several participants also described adapting their prompting strategies over time. P11 explained that they initially used “*normal human language*” with SkillComposer, but learned that the baseline required “*very specific prompts.*” P5 echoed this sentiment, noting that switching from repeated commands to a single high-level prompt “*made my life easier.*” These observations are consistent with the baseline’s higher prompt count and its lower initial success rate on the Rearrange Objects task, where participants were still learning how to prompt the system effectively.

Participants also suggested future improvements and applications. Several wanted improved accuracy and response time. Others were interested in applying SkillComposer to additional scenarios, such as household humanoid robots that retrieve objects or bimanual tasks like pouring wine into a cup as P9 suggested. Participants also suggested interface improvements, including a more visually attractive UI, adding voice interactions, a higher-frame-rate simulation preview,

and clearer status notifications. Together, these comments suggest that participants saw SkillComposer as a promising interface with potential for broader use.

VI. CONCLUSION

This paper presents SkillComposer, an interactive natural language robot programming system that combines evaluator-guided code generation with online skill learning. By validating generated programs before execution and learning reusable macros from accepted programs, SkillComposer enables the available robot skill set to evolve over time rather than remaining fixed. Ablation experiments showed that the generate-test loop substantially improved reliability, while macro learning reduced program length and enabled the reuse of learned skills across tasks. A user study further demonstrated that participants generally preferred SkillComposer over a baseline coding LLM, reporting that it better supported high-level, multi-step instructions and required fewer prompts to complete tasks. As the number of learned skills grows, the coder LLM’s ability to utilize these skills may degrade. Future work could address this by integrating retrieval-augmented generation to retrieve the most relevant learned skills from longer interaction histories, enabling long-term use with larger skill libraries. We believe that systems capable of continuously learning from user interaction can make robot programming more accessible by allowing users to describe tasks naturally while the system learns reusable skills tailored to their needs.

REFERENCES

- [1] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033, IEEE, 2012.
- [2] NVIDIA, “Isaac Sim.”
- [3] E. Rohmer, S. P. N. Singh, and M. Freese, “Coppeliassim (formerly v-rep): a versatile and scalable robot simulation framework,” in *Proc. of The International Conference on Intelligent Robots and Systems (IROS)*, 2013. www.coppeliarobotics.com.
- [4] A. Nanavati, V. Ranganeni, and M. Cakmak, “Physically assistive robots: A systematic review of mobile and manipulator robots that physically assist people with disabilities,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 7, 2023.
- [5] R. Ye, W. Xu, H. Fu, R. K. Jenamani, V. Nguyen, C. Lu, K. Dimitropoulou, and T. Bhattacharjee, “RCare World: A Human-centric Simulation World for Caregiving Robots,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 33–40, Oct. 2022. ISSN: 2153-0866.
- [6] S. H. Vemprala, R. Bonatti, A. Buckner, and A. Kapoor, “ChatGPT for Robotics: Design Principles and Model Abilities,” *IEEE Access*, vol. 12, pp. 55682–55696, 2024.
- [7] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, “ProgPrompt: program generation for situated robot task planning using large language models,” *Autonomous Robots*, vol. 47, pp. 999–1012, Dec. 2023.
- [8] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,” in *Conference on robot learning*, pp. 287–318, PMLR, 2023.
- [9] U. B. Karli, J.-T. Chen, V. N. Antony, and C.-M. Huang, “Alchemist: LLM-Aided End-User Development of Robot Applications,” in *Proceedings of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*, HRI ’24, (New York, NY, USA), pp. 361–370, Association for Computing Machinery, Mar. 2024.
- [10] Y. Ge, Y. Dai, R. Shan, K. Li, Y. Hu, and X. Sun, “Cocobo: Exploring Large Language Models as the Engine for End-User Robot Programming,” in *2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pp. 89–95, Sept. 2024.
- [11] M. Bowers, T. X. Olausson, L. Wong, G. Grand, J. B. Tenenbaum, K. Ellis, and A. Solar-Lezama, “Top-Down Synthesis for Library Learning,” *Proc. ACM Program. Lang.*, vol. 7, pp. 41:1182–41:1213, Jan. 2023.
- [12] L. Wang, Y. Ling, Z. Yuan, M. Shridhar, C. Bao, Y. Qin, B. Wang, H. Xu, and X. Wang, “Gensim: Generating robotic simulation tasks via large language models,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [13] D. G. Paul, H. Zhu, and I. Bayley, “Benchmarks and metrics for evaluations of code generation: A critical review,” in *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*, pp. 87–94, 2024.
- [14] V. N. Devarakonda, A. U. Kaypak, S. Yuan, P. Krishnamurthy, Y. Fang, and F. Khorrami, “Multitalk: Introspective and extrospective dialogue for human-environment-llm alignment,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 10737–10743, 2025.
- [15] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, “Large language models as tool makers,” in *International Conference on Learning Representations*, vol. 2024, pp. 54067–54089, 2024.
- [16] C. Qian, C. Han, Y. Fung, Y. Qin, Z. Liu, and H. Ji, “CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models,” in *Findings of the Association for Computational Linguistics: EMNLP 2023* (H. Bouamor, J. Pino, and K. Bali, eds.), (Singapore), pp. 6922–6939, Association for Computational Linguistics, Dec. 2023.
- [17] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An Open-Ended Embodied Agent with Large Language Models,” Oct. 2023. arXiv:2305.16291 [cs].
- [18] G. Tzifas and H. Kasaei, “Lifelong Robot Library Learning: Bootstrapping Composable and Generalizable Skills for Embodied Control with Language Models,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 515–522, May 2024.
- [19] M. Parakh, A. Fong, A. Simeonov, T. Chen, A. Gupta, and P. Agrawal, “Lifelong Robot Learning with Human Assisted Language Planners,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 523–529, May 2024.
- [20] Y. Meng, Z. Sun, M. Fest, X. Li, Z. Bing, and A. Knoll, “Growing with Your Embodied Agent: A Human-in-the-Loop Lifelong Code Generation Framework for Long-Horizon Manipulation Skills,” Sept. 2025. arXiv:2509.18597 [cs].
- [21] K. Ellis, L. Wong, M. Nye, M. Sablé-Meyer, L. Cary, L. Anaya Pozo, L. Hewitt, A. Solar-Lezama, and J. B. Tenenbaum, “DreamCoder: growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning,” *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 381, p. 20220050, June 2023.
- [22] L. Wong, K. M. Ellis, J. Tenenbaum, and J. Andreas, “Leveraging Language to Learn Program Abstractions and Search Heuristics,” in *Proceedings of the 38th International Conference on Machine Learning*, pp. 11193–11204, PMLR, July 2021.
- [23] G. Grand, L. Wong, M. Bowers, T. X. Olausson, M. Liu, J. B. Tenenbaum, and J. Andreas, “Lilo: Learning interpretable libraries by compressing and documenting code,” in *International Conference on Learning Representations*, vol. 2024, pp. 30399–30446, 2024.
- [24] S. Chen, R. Ye, S. Dingwani, P. Fazli, H. Seifi, and T. Bhattacharjee, “RCareGen: An Interface for Scene and Task Generation in RCare-World,” in *2025 20th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pp. 1274–1278, Mar. 2025.
- [25] J. Brooke, “SUS: A ‘Quick and Dirty’ Usability Scale,” in *Usability Evaluation In Industry*, CRC Press, 1996. Num Pages: 6.
- [26] A. Bangor, P. T. Kortum, and J. T. Miller, “An Empirical Evaluation of the System Usability Scale,” *International Journal of Human-Computer Interaction*, vol. 24, pp. 574–594, July 2008.